

República Federativa do Brasil

Ministério da Educação

Universidade Federal do Amazonas

Instituto de Computação

Linguagens de Programação 2026-1

Lista de Exercícios II: Programação OO, Meta-programação, Agentes e LLMs, v 1.0

Professor Dr. Marco Cristo

**Alunos:** Nome Sobrenome, Nome Sobrenome e Nome Sobrenome  
**E-mail:** nome.sobrenome@icomp.ufam.edu.br, nome.sobrenome@icomp.ufam.edu.br e nome.sobrenome@icomp.ufam.edu.br

PARTE I: Programação OO com Kotlin

1.1. Finalizando Curso Básico de Kotlin

Faça as últimas duas lições do tutorial de Kotlin em: <https://kotlinlang.org/docs/kotlin-tour-hello-world.html>. Depois, usando um estilo OO em Kotlin, implemente uma classe imutável `Matriz2D` que representa uma matriz de números inteiros de dimensão  $m \times n$ .

Inicialização

```
// Construtor deve ser validado: matriz não pode ser vazia e todas as linhas devem ter o mesmo tamanho.  
val a = Matriz2D(listOf(listOf(1, 2), listOf(3, 4)))  
Operadores sobrecarregados e Métodos
```

| Operação                                          | Sintaxe/Exemplo           | Comportamento                                                                                                                                |
|---------------------------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Soma                                              | <code>a + b</code>        | Soma elemento a elemento. Lança <code>IllegalArgumentException</code> se as dimensões forem diferentes.                                      |
| Subtração                                         | <code>a - b</code>        | Subtração elemento a elemento. Lança <code>IllegalArgumentException</code> se as dimensões forem diferentes.                                 |
| Multiplicação por constante (direita)             | <code>a * 2</code>        | Multiplica todos os elementos por 2.                                                                                                         |
| Multiplicação por constante (esquerda)            | <code>2 * a</code>        | Mesmo comportamento que <code>a * 2</code> .                                                                                                 |
| Multiplicação matricial                           | <code>a * b</code>        | Produto matricial. Lança <code>IllegalArgumentException</code> se <code>a.colunas != b.linhas</code> .                                       |
| Comparação                                        | <code>a == b</code>       | Compara conteúdo, não identidade. Não use data class.                                                                                        |
| Acesso via <code>get(i: Int, j: Int)</code>       | <code>a[0, 1]</code>      | Retorna o elemento na linha i, coluna j. Lança <code>IndexOutOfBoundsException</code> se índices inválidos.                                  |
| Conversão para String via <code>toString()</code> | <code>println(a)</code>   | Retorna cada linha entre colchetes, separada por vírgula, e as linhas separadas por <code>\n</code> . Exemplo: <code>[[1, 2], [3, 4]]</code> |
| Transposição matricial                            | <code>a.transpor()</code> | Retorna uma nova matriz com linhas e colunas trocadas.                                                                                       |

Dicas de implementação

- Estrutura básica

```

class Matriz2D(dados: List<List<Int>>) {
    private val dados: List<List<Int>> = dados.map { it.toList() }

    val linhas: Int = this.dados.size
    val colunas: Int = this.dados.first().size

    init {
        require(this.dados.isNotEmpty()) { "Matriz não pode ser vazia" }
        val primeiraLinha = this.dados.first().size
        require(this.dados.all { it.size == primeiraLinha }) {
            "Todas as linhas devem ter o mesmo tamanho"
        }
    }

    // Operadores
    operator fun plus(outra: Matriz2D): Matriz2D
    operator fun minus(outra: Matriz2D): Matriz2D
    operator fun times(escalar: Int): Matriz2D
    operator fun times(outra: Matriz2D): Matriz2D

    // Métodos
    fun transpor(): Matriz2D
    operator fun get(i: Int, j: Int): Int
    override fun toString()
    override fun equals(other: Any?): Boolean
}

```

- Multiplicação por constante à esquerda: fora da classe, você deve sobrecarregar o operador `times` da classe `Int`

```
operator fun Int.times(matriz: Matriz2D): Matriz2D
```

- Casos de teste sugeridos
  - Exibição correta de matrizes
  - Soma/subtração de matrizes 2x2
  - Multiplicação matricial 2x2 x 2x2
  - Multiplicação por constante (esquerda e direita)
  - Transposição de matriz 2x3
  - Acesso a elemento
  - Comparação de matrizes
  - Exceção para multiplicação com dimensões incompatíveis
  - Exceção para matriz com linhas de tamanhos diferentes

#### Exemplo não exaustivo de uso esperado

```

fun main() {
    val a = Matriz2D(listOf(listOf(1, 2), listOf(3, 4)))
    val b = Matriz2D(listOf(listOf(0, 1), listOf(1, 0)))

    println(a + b)      // [[1, 3], [4, 4]]
    println(a - b)      // [[1, 1], [2, 4]]
    println(a * b)      // [[2, 1], [4, 3]]
    println(2 * a)       // [[2, 4], [6, 8]]
    println(a * 3)       // [[3, 6], [9, 12]]
    println(a.transpor()) // [[1, 3], [2, 4]]
    println(a[0, 1])     // 2

    // Exceções:
    // Matriz2D(listOf(listOf(1, 2), listOf(3))) // IllegalArgumentException
    // a * Matriz2D(listOf(listOf(1, 2, 3)))     // IllegalArgumentException
    // Matriz2D(listOf(listOf(1))).det()         // IllegalArgumentException (não é
2x2)
}

```

In [ ]:

## 1.2. Comparação Kotlin vs Python

Com base no que você aprendeu sobre Kotlin OO e na sua experiência com Python OO, compare as duas linguagens considerando os seguintes critérios:

- Legibilidade, Escrita, Confiabilidade
- Simplicidade, Ortogonalidade, Tipos de Dados, Projeto de Sintaxe, Suporte à Abstração, Expressividade, Checagem de Tipos, Manipulação de Exceções e Restrição de Aliases

Escreva um parágrafo para cada critério, destacando semelhanças e diferenças entre Kotlin e Python no contexto da programação OO.

In [ ]:

## PARTE II: Interface de Linha de Comando com Meta-programação em Python

Vamos implementar uma versão simplificada da biblioteca `Fire`, que transforma funções Python comuns em comandos de uma interface de linha de comando (CLI). Para isso, você deve usar meta-programação para criar uma ferramenta que automaticamente gera uma interface CLI a partir de qualquer arquivo Python.

**Objetivo:** implementar uma função `meu_fire()` que:

1. Recebe o caminho de um arquivo Python como argumento.
2. Carrega o módulo dinamicamente.
3. Identifica todas as funções definidas no nível global do módulo.
4. Transforma cada função em um subcomando da CLI.
5. Realiza o parsing automático de argumentos (posicionais e nomeados) com base na assinatura da função.
6. Extrai e exibe docstrings para gerar ajuda automática.
7. Executa a função solicitada com os argumentos fornecidos.

### Exemplo Completo de Uso

**Arquivo exemplo.py:**

```
"""Módulo de exemplo com operações matemáticas e utilitárias."""

def ola(nome: str, saudacao="Olá"):
    """Exibe uma saudação personalizada.

    Args:
        nome: Nome da pessoa a ser saudada.
        saudacao: Palavra de saudação (padrão: 'Olá').
    """
    print(f"{saudacao}, {nome}!")

def soma(a: int, b: int = 0) -> int:
    """Retorna a soma de dois números inteiros.

    Args:
        a: Primeiro número
        b: Segundo número (padrão = 0)

    Returns:
        Soma calculada
    """
    resultado = a + b
    print(resultado)
    return resultado

def ativo(debug: bool = False):
    """Controla o modo de depuração do sistema."""
    print("Debug ativado" if debug else "Modo normal")
```

**Execuções no terminal:**

```
$ python meu_fire.py exemplo.py --help
```

```
Módulo: exemplo.py
```

```
Descrição: Módulo de exemplo com operações matemáticas e utilitárias.
```

Comandos disponíveis:

`ola` - Exibe uma saudação personalizada.

Parâmetros:

--nome (str, obrigatório): Nome da pessoa a ser saudada.

--saudacao (str, padrão='Olá'): Palavra de saudação.

`soma` - Retorna a soma de dois números inteiros.

Parâmetros:

--a (int, obrigatório): Primeiro número

--b (int, padrão=0): Segundo número

`ativo` - Controla o modo de depuração do sistema.

Parâmetros:

--debug (bool, padrão=False)

```
$ python meu_fire.py exemplo.py ola --nome Ana
Olá, Ana!
```

```
$ python meu_fire.py exemplo.py soma 10 --b 20
30
```

```
$ python meu_fire.py exemplo.py ativo --debug
Debug ativado
```

```
$ python meu_fire.py exemplo.py soma --help
```

Comando: `soma`

Descrição: Retorna a soma de dois números inteiros.

Uso: `soma --a INT [--b INT]`

Parâmetros:

--a (int, obrigatório): Primeiro número

--b (int, padrão=0): Segundo número

Retorna: Soma calculada

## Requisitos Obrigatórios

### 1. Mapeamento automático de funções

- `meu_fire(arquivo.py)` deve ler **todas as funções definidas no nível global** do módulo.
- Funções importadas de outros módulos **não** devem ser incluídas.
- Funções internas (dentro de outras funções) **não** devem ser incluídas.
- Classes e seus métodos estão **fora do escopo** (ignorar).
- Cada função se torna um subcomando com o mesmo nome da função.

**2. Parsing de argumentos posicionais e nomeados:** suporte a `int`, `float`, `str`, `bool` (aceita `True`, `true`, `yes` ou 1 como `True`; `False`, `false`, `no` ou 0 como `False`)

| Tipo de argumento                                | Comportamento na CLI                                                           |
|--------------------------------------------------|--------------------------------------------------------------------------------|
| Sem valor padrão ( <code>def f(a)</code> )       | Parâmetro <b>posicional obrigatório</b>                                        |
| Com valor padrão ( <code>def f(a, b=10)</code> ) | Parâmetro <b>nomeado opcional</b> (ex: <code>--b 20</code> )                   |
| <code>bool</code> com padrão <code>False</code>  | Flag: <code>--debug</code> → <code>True</code> , ausência → <code>False</code> |
| <code>bool</code> com padrão <code>True</code>   | Flag: <code>--debug false</code> → <code>False</code>                          |

### 3. Geração automática de ajuda

- Ajuda geral (comando sem função ou com `--help/ajuda`): listar todos os comandos disponíveis.
- Para cada comando, mostrar:
  - Nome da função.
  - Breve descrição (primeira linha do docstring).
  - Parâmetros (nome, tipo, obrigatoriedade, valor padrão se houver).
  - Ajuda específica (bônus recomendado):
- Extração e exibição de docstrings

- Docstring do módulo: Se o arquivo .py tiver um docstring no topo, ele deve ser exibido no cabeçalho do help geral.
- Docstring da função devem aceitar os formatos Mínimo e Google:

```
# formato mínimo
def multiplica(a: int, b: int) -> int:
    """Multiplica a por b. Parâmetros: a (int), b (int)."""
    return a * b

# estilo Google
def soma(a: int, b: int) -> int:
    """Retorna a soma de dois números.

    Args:
        a: Primeiro número inteiro.
        b: Segundo número inteiro.

    Returns:
        A soma de a e b.
    """
    return a + b
```

#### 4. Tratamento de erros básico

- Arquivo não encontrado: Erro: Arquivo 'X' não encontrado
- Função inexistente: Erro: Comando 'X' não encontrado. Use --help para listar.
- Tipo de argumento inválido: Erro: Parâmetro 'a' esperava int, recebeu 'abc'
- Número incorreto de argumentos: Erro: Função 'soma' requer 2 argumentos posicionais

#### Restrições e Fora do Escopo

Ao contrário do Fire original, **não** é para implementar:

- Suporte a classes, métodos ou objetos.
- Aninhamento de comandos (cli modulo func1 func2).
- Parsing de argumentos com aspas para strings contendo espaço.
- `*args` (argumentos variádicos posicionais) e `**kwargs` (argumentos nomeados variádicos).
- Listas, dicionários ou tipos compostos como parâmetros.
- Execução assíncrona ( `async def` ).
- Decoradores complexos.

#### Liberdade para o aluno:

- Pode usar `argparse` da biblioteca padrão desde que os argumentos sejam gerados dinamicamente via introspecção.
- Pode implementar o parser manualmente com `sys.argv` .

#### Dicas de Implementação

Bibliotecas úteis (todas da biblioteca padrão):

- `inspect` : `signature()`, `getmembers()`, `isfunction()`, `getdoc()`, `getmodule()`
- `importlib.util` : carregar módulo dinamicamente de um arquivo .py
- `sys` : acessar argumentos da linha de comando
- `argparse` : parser de argumentos (opcional)
- `re` : expressões regulares para parsear docstrings

#### Entrega

**Formato:** Um único arquivo `meu_fire.py` implementando a função `meu_fire()` e o código para executá-la via linha de comando.

#### Estrutura esperada:

```
#!/usr/bin/env python3
"""Implementação de mini-Fire."""

import sys
```

```

import inspect
import importlib.util
# outros imports necessários

def meu_fire(caminho_modulo):
    # implementação principal
    pass

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Uso: python meu_fire.py <arquivo.py> [comando] [args...]")
        sys.exit(1)

    caminho = sys.argv[1]
    meu_fire(caminho)

```

**Testes Rápidos:** para você validar antes de entregar

- Teste básico: `python meu_fire.py exemplo.py ola --nome "João"`
- Teste de help: `python meu_fire.py exemplo.py --help`
- Teste de erro de tipo: `python meu_fire.py exemplo.py soma 10 --b "texto"` (Deve reclamar que b deveria ser int)
- Teste de docstring: verifique se a descrição de soma aparece com os parâmetros documentados.

In [ ]:

## PARTE III: Nota Secreta

### 3.1. Objetivos

Neste trabalho vocês devem:

1. **Implementar agentes inteligentes** que se comunicam via protocolo A2A (Agent-to-Agent)
2. **Integrar LLMs locais** a agentes autônomos através de um serviço centralizado, similar a APIs como OpenAI/Google
3. **Compreender os desafios** de sistemas distribuídos: comunicação remota, concorrência, filas de requisição
4. **Projetar estratégias** para diferentes tipos de agentes (criativo vs. estratégico)
5. **Comparar o desempenho** de diferentes abordagens em um ambiente competitivo

**Nota:** Diferentes componentes do *software* a ser implementado (o *Game Master*, o serviço LLM centralizado e um agente baseline aleatório) serão **fornecidos** pelo professor. Seu grupo deve fornecer a implementação de **um agente inteligente** que vocês julguem seja o mais adequado para uma avaliação competitiva.

### 3.2. O Jogo: Nota Secreta

**Nota Secreta** é uma adaptação do famoso jogo Dixit para o universo da música brasileira. Em vez de cartas com imagens abstratas, os jogadores utilizam **letras de músicas brasileiras** como elementos do jogo.

O jogo envolve **6 agentes autônomos** (implementados por você e fornecidos pelo professor) que:

- Recebem uma "mão" de 4 letras de música cada
- Alternam-se como "narrador" (quem dá a dica) e "melômanos" (amantes de músicas que tentam adivinhar as canções)
- Usam um serviço LLM centralizado para tomar decisões
- Competem para atingir **30 pontos**

#### Regras Completas

Cada rodada funciona assim:

1. **Distribuição:** Cada agente recebe 4 músicas (para reduzir custo de comunicação e inferência, cada música enviada ao agente contém uma visão enxuta da carta, com pelo menos id, title e uma versão truncada da letra).
2. **Narrador:** Um agente é escolhido como narrador da rodada (ordem circular)
3. **Escolha da letra:** O narrador seleciona UMA das suas 4 músicas

- 4. **Dica:** O narrador gera uma dica de **no máximo 6 palavras** sobre a música escolhida
- 5. **Seleção dos melômanos:** Cada outro agente escolhe, da sua própria mão, a música que MELHOR combina com a dica
- 6. **Votação:** Todas as 6 cartas (1 do narrador + 5 dos outros) são embaralhadas e apresentadas. Cada melômano deve votar em exatamente **duas** opções distintas. O narrador não vota. Nenhum agente pode votar na própria carta. Um melômetro acerta a carta do narrador se pelo menos um dos seus dois votos for a carta do narrador.
- 7. **Pontuação:** Segue as regras do Dixit (detalhadas abaixo)
- 8. **Reposição:** Novas músicas são distribuídas para manter 4 cartas por agente
- 9. **Próxima rodada:** O papel de narrador passa para o próximo agente

**Pontuação** (Regra Oficial do Dixit Odyssey)

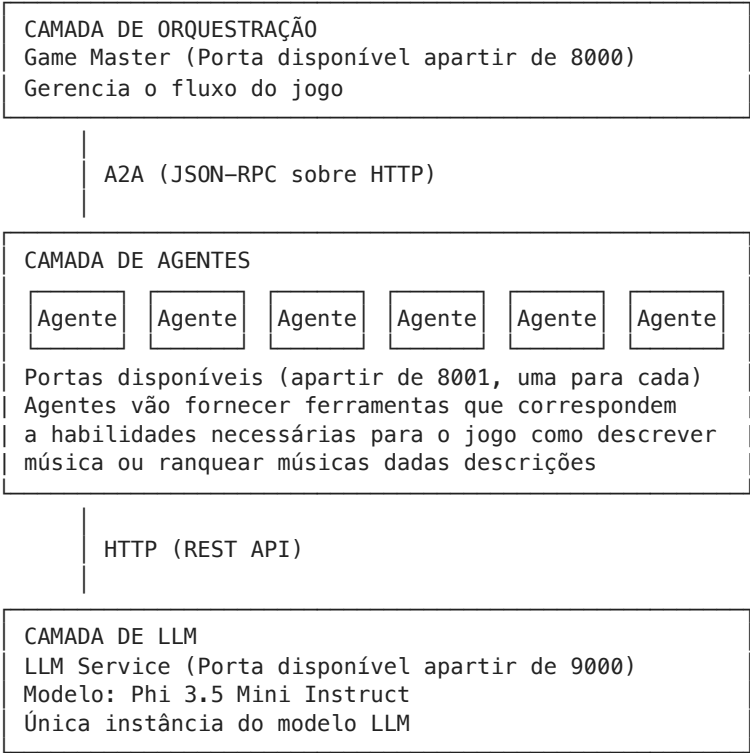
| Situação                           | Pontuação do narrador | Pontuação de cada um dos melômanos                                                               |
|------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
| Ninguém votou na carta do narrador | 0                     | +2 pela rodada +1 para cada voto recebido em sua própria letra (limitado a 3)                    |
| Todos votaram na carta do narrador | 0                     | +2 pela rodada +1 para cada voto recebido em sua própria letra (limitado a 3)                    |
| Caso contrário                     | +3                    | +3 se acertou a letra do narrador +1 para cada voto recebido em sua própria letra (limitado a 3) |

Observações:

- O melômano acerta a letra do narrador se ele a incluir entre os seus dois votos;
- O narrador pode ganhar 3 pontos por rodada, no cenário de sucesso (ele não ganha pontos por voto recebido)
- Um agente pode ganhar no máximo **6 pontos por rodada** (3 por acertar a dica do narrador e, no máximo, 3 por votos recebidos)
- O jogo termina quando um agente atinge **30 pontos**, se tornando o vencedor da partida.

3.3 Arquitetura do Sistema

**Visão Geral.** O sistema é composto por **três camadas**:



**Game Master (Fornecido e não deve ser modificado)**

O Game Master é o *coordenador central* que:

- Escuta por agentes via A2A (aguarda 6 agentes se registrarem)
- Gerencia a base de músicas brasileiras
- Distribui e repõe as mãos dos agentes
- Orquestra o fluxo do jogo (rodadas, narrador, votação)
- Calcula pontuações
- Fornece logs detalhados em JSON

### LLM Service (Fornecido e não deve ser modificado)

O LLM Service é um *servidor centralizado* que:

- Carrega um *único modelo LLM local* ( Phi 3.5 Mini Instruct )
- Expõe uma API REST para geração de texto
- Gerencia fila de requisições dos agentes
- Simula o modelo de APIs como OpenAI/Google

Ele é centralizado pois economiza memória (um modelo vs. vários modelos para cada agente), permite rodar em máquinas com apenas 8GB RAM e é mais realista (mesmo padrão de APIs comerciais).

### Agentes Jogadores (Você Implementa)

Cada agente é um *servidor A2A independente* que:

- Comunica-se com o Game Master via A2A (JSON-RPC)
- Consulta o LLM Service via REST
- Mantém seu próprio estado (mão de cartas, histórico)
- Implementa **5 ferramentas (tools)** obrigatórias

Espera-se que seu agente ( `llm_agent.py` ):

- Use a LLM no núcleo semântico de suas decisões, embora seja permitido — e recomendado — usar heurísticas auxiliares para:
  - validar respostas;
  - tratar timeouts;
  - fazer fallback;
  - garantir consistência do protocolo.
- *Objetivo*: Vencer o jogo consistentemente

## 3.4. Interface dos Agentes (A2A)

Cada agente deve expor **5 ferramentas** (tools) que o Game Master irá chamar:

### Tool 1: `receive_hand`

```
@app.tool()
async def receive_hand(hand: List[Dict]) -> Dict:
    """
    Recebe as 4 músicas do Game Master.

    Args:
        hand: Lista de 4 músicas, cada uma no formato:
            {
                "id": 1,
                "title": "Garota de Ipanema",
                "lyrics": "Olha que coisa mais linda...",
            }

    Returns:
        {"status": "ok", "hand_size": 4}
    """
```

### Tool 2: `choose_card`

```
@app.tool()
async def choose_card() -> Dict:
```



```

"""
Escolhe qual carta da mão usar como narrador.
(Chamado apenas quando o agente é o narrador)

Returns:
    {"chosen_card": <música completa>}
"""

```

#### Tool 3: send\_clue

```

@app.tool()
async def send_clue(lyrics: str, max_words: int = 6) -> Dict:
    """
    Gera uma dica de até max_words palavras.
    (Chamado apenas quando o agente é o narrador)

    Args:
        lyrics: A letra da música escolhida
        max_words: Número máximo de palavras permitido

    Returns:
        {"clue": "sua dica aqui (até 6 palavras)"}
    """

```

#### Tool 4: select\_card\_by\_clue

```

@app.tool()
async def select_card_by_clue(clue: str) -> Dict:
    """
    Escolhe qual carta da mão melhor representa a dica.
    (Chamado quando o agente NÃO é o narrador)

    Args:
        clue: A dica fornecida pelo narrador

    Returns:
        {"chosen_card": <música completa>}
    """

```

#### Tool 5: vote

```

@app.tool()
async def vote(clue: str, options: List[Dict], my_chosen_card: Dict) -> Dict:
    """
    Responsável por votar em quais opções podem ser a carta do narrador.
    É chamada apenas quando o agente NÃO é o narrador.

    Args:
        clue: A dica fornecida pelo narrador
        options: Lista de 6 cartas embaralhadas (0 a 5)
        my_chosen_card: A carta que este agente jogou nesta rodada

    Returns:
        {"votes": [i1, i2]} # i1 e i2 entre 0 a 5; i1 != i2; i1 e i2 não podem ser o
        índice da sua própria carta
    """

```

### 3.5. Como Usar o LLM Service

O serviço LLM pode operar em modo real (com um modelo local carregado) ou em modo *mock*, para testes de infraestrutura e depuração quando não houver modelo disponível.

#### Endpoint

POST <http://localhost:9000/generate>  
 Content-Type: application/json

#### Corpo da Requisição

```

{
    "prompt": "Seu prompt aqui",
    "max_tokens": 100,
    "temperature": 0.7,

```

```
"stop": ["\n", "###"]
}
```

#### Resposta

```
{
  "text": "Texto gerado pela LLM",
  "usage": {
    "prompt_tokens": 45,
    "completion_tokens": 23,
    "total_tokens": 68
  }
}
```

#### Exemplo de Uso no seu Agente

```
import aiohttp

class MeuAgente:
    def __init__(self):
        self.llm_url = "http://localhost:9000"

    async def llm_generate(self, prompt, max_tokens=100, temperature=0.7):
        async with aiohttp.ClientSession() as session:
            async with session.post(
                f"{self.llm_url}/generate",
                json={
                    "prompt": prompt,
                    "max_tokens": max_tokens,
                    "temperature": temperature
                }
            ) as resp:
                result = await resp.json()
                return result["text"]
```

### 3.6. O Que É Fornecido

| Recurso                     | Descrição                                                                                                                                                                                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>game_master.py</code> | <ul style="list-style-type: none"> <li>- Coordenador da partida</li> <li>- Gerencia registro de agentes</li> <li>- Executa o jogo completo</li> <li>- Gera logs em <code>logs/partida_YYYYMMDD_HHMMSS.json</code></li> </ul>                                        |
| <code>llm_service.py</code> | <ul style="list-style-type: none"> <li>- Serviço LLM centralizado em FastAPI</li> <li>- Expõe <code>endpoints</code> <code>/generate</code> (para inferência) e <code>/health</code> para status</li> <li>- Pode rodar em modo real ou <code>mock</code></li> </ul> |

| `random_agent.py` | - Implementação completa de um agente aleatório

- Serve como template e baseline para comparação | `brazilian_songs.csv` | 256 músicas brasileiras com:
- `id` : Identificador único
- `title` : Título da música
- `artist` : Artista/Compositor
- `lyrics` : letra. || `run_game.py` | - Script que orquestra toda a execução
- Pode escolher portas livres automaticamente || `base_agent.py` | - Classe base com cliente LLM integrado
- Métodos auxiliares para formatação
- Tratamento de erros básico | `fast_a2a.py` | - Mini-camada A2A que implementa:
- `A2AApp` para subir o servidor do agente
- `@tool` para marcar tools expostas remotamente
- endpoint `/rpc` com JSON-RPC sobre HTTP | `render_log_readable.py` | - Script auxiliar para leitura de logs

### 3.7. O Que Você Deve Implementar

**Agente Estratégico (`llm_agent.py`)** Implemente um agente que combina heurísticas + LLM Service:

```
from base_agent import BaseAgent
from fasta2a import A2AApp, tool

app = A2AApp(name="LLMAgent")
```

```

class LLMAgent(BaseAgent):
    def __init__(self, name: str, llm_url: str):
        super().__init__(name=name, llm_url=llm_url)
        self.hand = []
        self.vote_history = []
        self.clue_history = []

    @tool()
    async def receive_hand(self, hand):
        self.hand = list(hand)
        return {"status": "ok", "hand_size": len(self.hand)}

    @tool()
    async def choose_card(self):
        # heurística e/ou apoio da LLM
        ...

    @tool()
    async def send_clue(self, lyrics: str, max_words: int = 6):
        # uso da LLM + sanitização da resposta
        ...
        return {"clue": clue}

    @tool()
    async def select_card_by_clue(self, clue: str):
        # escolha de carta guiada por dica
        ...
        return {"chosen_card": chosen_card}

    @tool()
    async def vote(self, clue: str, options: list, my_chosen_card: dict):
        # deve retornar exatamente dois votos distintos
        ...
        return {"votes": [i1, i2]}

```

### 3.8. Setup e Execução

#### Requisitos de Hardware

- RAM: 8GB (mínimo) ou 16GB (recomendado)
- CPU: 4+ cores (qualquer processador moderno)
- GPU: Não obrigatória (usaremos CPU com modelo quantizado)
  - Em máquinas sem GPU, modelos pequenos podem apresentar alta latência e respostas inconsistentes. Portanto, espera-se que os agentes usem prompts curtos, *parsing* robusto, *fallback* e estratégias de contenção de custo.
- Disco: 5GB livres (para o modelo LLM)

#### Instalação

# 1. Clone o repositório (ou crie seu ambiente)

```
git clone <url-fornecida-pelo-professor>
cd nota-secreta
```

# 2. Crie ambiente virtual

```
python -m venv venv
source venv/bin/activate # Linux/Mac
# ou
venv\Scripts\activate # Windows
```

# 3. Instale dependências

```
pip install -r requirements.txt
```

# 4. Baixe o modelo LLM (Phi-3.5-mini-instruct-GGUF)

```
wget https://huggingface.co/bartowski/Phi-3.5-mini-instruct-GGUF/blob/main/Phi-3.5-mini-instruct-Q4_K_M.gguf
```

#### Execução Completa

```
# -----
# Comando único para subir toda a arquitetura e executar uma partida completa
# -----
python run_game.py --model Phi-3.5-mini-instruct-Q4_K_M.gguf

# Observação:
# Se as portas padrão estiverem ocupadas, o script pode escolher portas livres
# automaticamente.
# O script também sobe:
# - o serviço LLM
# - o Game Master
# - 1 agente estratégico
# - 5 agentes aleatórios
# e ao final dispara uma partida completa.
# -----
# Outras opções
# --force-mock: não usa serviço da LLM. Apenas simula uso.
# --all-strategic: sobe 6 agentes estratégicos
# --llm-max-concurrency N: usa N cores concorrentemente

# -----
# Execução manual, passo a passo
# -----
# Terminal 1: LLM Service
python llm_service.py --model Phi-3.5-mini-instruct-Q4_K_M.gguf --port 9000 --max-
concurrency 1

# Terminal 2: Game Master
python game_master.py --port 8000 --db brazilian_songs.csv --target-score 30 --log-dir
logs

# Terminais 3-8: agentes
python llm_agent.py http://127.0.0.1:8000 --port 8001 --llm-url http://127.0.0.1:9000 -
-name LLMAgent_1
python random_agent.py http://127.0.0.1:8000 --port 8002 --llm-url
http://127.0.0.1:9000 --name RandomAgent_2
python random_agent.py http://127.0.0.1:8000 --port 8003 --llm-url
http://127.0.0.1:9000 --name RandomAgent_3
python random_agent.py http://127.0.0.1:8000 --port 8004 --llm-url
http://127.0.0.1:9000 --name RandomAgent_4
python random_agent.py http://127.0.0.1:8000 --port 8005 --llm-url
http://127.0.0.1:9000 --name RandomAgent_5
python random_agent.py http://127.0.0.1:8000 --port 8006 --llm-url
http://127.0.0.1:9000 --name RandomAgent_6

# Depois de subir os processos manualmente, registre os agentes no Game Master
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "LLMAgent_1", "url": "http://127.0.0.1:8001", "kind": "strategic"}'
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "RandomAgent_2", "url": "http://127.0.0.1:8002", "kind": "random"}'
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "RandomAgent_3", "url": "http://127.0.0.1:8003", "kind": "random"}'
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "RandomAgent_4", "url": "http://127.0.0.1:8004", "kind": "random"}'
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "RandomAgent_5", "url": "http://127.0.0.1:8005", "kind": "random"}'
curl -X POST http://127.0.0.1:8000/register -H "Content-Type: application/json" -d
'{"name": "RandomAgent_6", "url": "http://127.0.0.1:8006", "kind": "random"}'

# Por fim, peça ao Game Master para executar a partida
curl -X POST http://127.0.0.1:8000/play
O run_game.py é seu amigo. Ele faz tudo isso por você automaticamente:
```

- Inicia o LLM Service (aguarda modelo carregar)
- Inicia o Game Master
- Inicia instância do seu StrategicAgent
- Inicia 5 instâncias do RandomAgent (fornecido)
- Aguarda o fim do jogo (alguém atinge 30 pontos)
- Exibe placar final e estatísticas

- Encerra todos os processos

## Logs e Análise

O Game Master gera um arquivo JSON por partida em `logs/` :

```
{
  "timestamp": "2026-04-19T17:59:33.794794",
  "agents": [{
    "id": 0, "type": "strategic", "name": "LLMAgent_1",
    "id": 1, "type": "random", "name": "RandomAgent_2",
    "id": 2, "type": "random", "name": "RandomAgent_3",
    "id": 3, "type": "random", "name": "RandomAgent_4",
    "id": 4, "type": "random", "name": "RandomAgent_5",
    "id": 5, "type": "random", "name": "RandomAgent_6"
  }],
  "initial_hands": [
    {
      "agent": 0, "agent_name": "LLMAgent_1",
      "cards": [
        {
          "id": 19, "title": "Alagados", "artist": "Os Paralamas do Sucesso",
          "lyrics": "Todo dia o sol da ..."
        },
        {
          "id": 5, "title": "Alegria, Alegria", "artist": "Caetano Veloso",
          "lyrics": "Caminhando contra o ..."
        },
        ...
      ]
    },
    {
      "agent": 1, "agent_name": "RandomAgent_2",
      "cards": [
        {
          "id": 108, "title": "É o Amor", "artist": "Zezé Di Camargo & Luciano",
          "lyrics": "Eu não vou negar ..."
        },
        ...
      ]
    },
    ...
  ],
  "rounds": [
    {
      "round": 1, "narrador": 0, "narrador_option": 2, "clue": "não quando joão pra vida nem",
      "played_cards": [
        {
          "option": 0, "agent": 1, "agent_name": "RandomAgent_2", "card_id": 108,
          "title": "É o Amor", "artist": "Zezé Di Camargo & Luciano",
          "lyrics": "Eu não vou negar que sou ..."
        },
        {
          "option": 1, "agent": 5, "agent_name": "RandomAgent_6", "card_id": 42,
          "title": "Conversa de Botequim", "artist": "Noel Rosa",
          "lyrics": "Seu garçom\nFaça o favor de ..."
        },
        {
          "option": 2, "agent": 0, "agent_name": "LLMAgent_1", "card_id": 100,
          "title": "João Valentão", "artist": "Dorival Caymmi",
          "lyrics": "João Valentão é brigão..."
        },
        {
          "option": 3, "agent": 3, "agent_name": "RandomAgent_4", "card_id": 52,
          "title": "Epitáfio", "artist": "Titãs",
          "lyrics": "Devia ter amado mais ..."
        },
        {
          "option": 4, "agent": 2, "agent_name": "RandomAgent_3", "card_id": 11,
          "title": "Roda Viva", "artist": "Chico Buarque",
          "lyrics": "Tem dias que a gente se sente..."
        },
        {
          "option": 5, "agent": 4, "agent_name": "RandomAgent_5", "card_id": 157,
          "title": "Gita", "artist": "Raul Seixas",
          "lyrics": "Eu que já andei pelos quatro cantos do mundo procurando..."
        }
      ]
    },
    ...
  ],
  "votes": [[], [3, 4], [3, 0], [5, 2], [2, 1], [3, 2]],
  "votes_by_agent": [
```

```

    { "agent": 0, "agent_name": "LLMAgent_1", "own_option": 2, "own_card_id": 100,
      "voted_options": [], "voted_agents": []},
    { "agent": 1, "agent_name": "RandomAgent_2", "own_option": 0, "own_card_id":
108,
      "voted_options": [3, 4], "voted_agents": [3, 2]},
    { "agent": 2, "agent_name": "RandomAgent_3", "own_option": 4, "own_card_id":
11,
      "voted_options": [3, 0], "voted_agents": [3, 1]},
    { "agent": 3, "agent_name": "RandomAgent_4", "own_option": 3, "own_card_id":
52,
      "voted_options": [5, 2], "voted_agents": [4, 0]},
    { "agent": 4, "agent_name": "RandomAgent_5", "own_option": 5, "own_card_id":
157,
      "voted_options": [2, 1], "voted_agents": [0, 5]
    },
    { "agent": 5, "agent_name": "RandomAgent_6", "own_option": 1, "own_card_id":
42,
      "voted_options": [3, 2], "voted_agents": [3, 0]}
  ],
  "received_votes": [1, 1, 3, 3, 1, 1],
  "received_votes_by_agent": {"1": 1, "5": 1, "0": 3, "3": 3, "2": 1, "4": 1},
  "scores": [3, 1, 1, 6, 4, 4],
  "cumulative_scores": [3, 1, 1, 6, 4, 4],
  "drawn_cards": [
    { "agent": 0, "agent_name": "LLMAgent_1", "card_id": 28, "title": "Apesar
de Você"},
    { "agent": 1, "agent_name": "RandomAgent_2", "card_id": 129, "title": "Uma
Brasileira"},
    { "agent": 2, "agent_name": "RandomAgent_3", "card_id": 205, "title": "BR-3"},
    { "agent": 3, "agent_name": "RandomAgent_4", "card_id": 162, "title": "É
Preciso Saber Viver"},
    { "agent": 4, "agent_name": "RandomAgent_5", "card_id": 55, "title":
"Ideologia"},
    { "agent": 5, "agent_name": "RandomAgent_6", "card_id": 231, "title":
"Desabafo"}
  ]
}, ...
],
"final_scores": [30, 25, 28, 22, 18, 20],
"winner": 0,
"total_rounds": 12

```

Notas:

- `votes[j]` são os votos dados pelo agente `j` (para o narrador é sempre uma lista vazia);
- `received_votes[k]` é o número de votos recebidos pela opção embaralhada `k` na rodada;
- Para mais legibilidade, há o campo derivado `received_votes_by_agent`.
- Este log foi feito para leitura e depuração e está sujeito a mudanças com a inclusão de campos extras.
- Desde que estes logs são de leitura complexa, é também fornecido o script `render_log_readable.py`, que traduz os logs para uma versão resumida mais apropriada para análise do jogo completo.

### 3.9. Entrega

- Arquivos a Entregar
  - Código fonte: `llm_agent.py` (seu agente criativo) e, talvez, `base_agent.py`.
  - README.md:
    - Instruções de instalação (se diferentes das fornecidas)
    - Como executar seu código
    - Exemplo de saída esperada (pelo menos 2 rodadas completas)
    - Descrição dos prompts e heurísticas implementadas no agente estratégico
    - Dificuldades encontradas e soluções
    - Integrantes do grupo
- Não é necessário entregar
  - O modelo LLM
  - A base de músicas (`brazilian_songs.csv`)
  - Logs gerados

Componentes fornecidos pelo professor podem ser modificados localmente para testes, mas a avaliação priorizará a compatibilidade da interface com a infraestrutura definida no enunciado.

### 3.10. Avaliação

| Critério   | Pontos | Descrição                                                                                                                                                                         |
|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Corretude  | 3      | - Qualidade da estratégia implementada, incluindo uso adequado da LLM<br>- Organização, comentários, <i>type hints</i> , <i>async/await</i> correto<br>- README claro e completo. |
| Robustez   | 2      | - Tratamento de erros & Logging<br>- Robustez a demora, respostas mal formadas ou ausência de respostas                                                                           |
| Protocolo  | 1      | - Agentes funcionam corretamente com Game Master fornecido, implementam todas as 5 tools                                                                                          |
| Desempenho | 1-4    | - Desempenho em competição envolvendo os agentes da turma                                                                                                                         |

### 3.11. Dicas e Boas Práticas

#### Performance e Otimização

- Mantenha `max_tokens` baixo (20-35 para respostas curtas)
- Use *temperature* adequadamente: valor baixo para decisões menos criativas mas mais precisas
- *Timeout* nas requisições: use `asyncio.wait_for` com 60 segundos
- *Fallback* para decisões: se LLM falhar, tenha comportamento padrão (ex: escolher primeira carta)
- Implemente cache para prompts repetidos (ex: mesma música sendo avaliada)

#### Debugging

```
# Adicione logs no seu agente
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def choose_card(self):
    logger.info(f"Escolhendo carta entre {len(self.hand)} opções")
    # ... lógica
    logger.info(f"Carta escolhida: {chosen['title']}")
```

#### Estrutura de Arquivos Esperada

```
nota-secreta/
├── llm_agent.py           # seu agente principal
├── base_agent.py         # fornecido; pode ser modificado/estendido
├── game_master.py        # fornecido; coordena o jogo e gera logs
├── llm_service.py        # fornecido; serviço LLM centralizado
├── random_agent.py       # fornecido; baseline aleatório
├── run_game.py           # fornecido; sobe toda a arquitetura
├── fasta2a.py            # fornecido: não modificar
├── render_log_readable.py # fornecido: pode modificar para depuração
├── brazilian_songs.csv   # fornecido
├── requirements.txt      # fornecido
├── logs/                 # gerado
├── tests/                # opcional
└── README.md            # sua documentação
```

#### Q&A

- Posso usar outro modelo LLM além do **Phi3.5-Mini-Instruct** ?
  - Não, o agente deve funcionar com o serviço LLM centralizado fornecido. Na avaliação, será usado este mesmo modelo.
- Posso usar bibliotecas e ferramentas de apoio ao uso de LLM, além das que foram usadas no código recebido, como LangChain?
  - Não, pois isso pode tornar extremamente difícil garantir que todos os agentes que vocês vão entregar rodem no ambiente virtual em que a competição será jogada. Eu prefiro manter os requirements restritos ao do ambiente dado para evitar problemas de conflitos entre diferentes requisitos de diferentes equipes.

- E se o **LLM Service** demorar muito?
  - O **Game Master** usa *timeouts* para evitar travamentos. Seu agente deve ser robusto a demora do serviço LLM, respostas mal formatadas e ausência de resposta útil. Ou seja, é aconselhável que você implemente estratégias como cache de prompts repetidos, *fallback* heurístico, *parsing* tolerante de respostas e *logging*.
- Preciso implementar concorrência no agente?
  - Não, o **Game Master** chama as *tools* sequencialmente. Seu agente processa uma requisição por vez.
- Posso modificar o **base\_agent.py** fornecido?
  - Sim, pode estender e modificar à vontade. Apenas mantenha a interface das 5 **tools**.
- Posso usar *threads* para paralelizar chamadas ao LLM?
  - Não é necessário. O **LLM Service** já gerencia fila. Múltiplas requisições simultâneas serão enfileiradas.
  - O serviço LLM centralizado pode serializar ou limitar a concorrência das requisições. Portanto, o agente não deve assumir respostas instantâneas nem paralelismo irrestrito.
- Como será a avaliação competitiva?
  - A avaliação competitiva será baseada em um torneio com múltiplas partidas, de modo a reduzir o efeito do acaso de uma única execução. Além dos agentes dos alunos, o torneio também incluirá um agente aleatório (baseline mínimo) e um agente de referência (define patamar de bom desempenho).
  - Para cada agente do aluno, será calculado seu desempenho médio no torneio. Também serão calculados:
    - DAA: desempenho médio do agente aleatório;
    - DAR: desempenho médio do agente de referência.
  - A pontuação competitiva será atribuída da seguinte forma, considerando comparações estatísticas:
    - 0 pt: agente com desempenho similar ao agente aleatório;
    - 1 pt: agente com desempenho melhor que o aleatório, mas abaixo do agente de referência;
    - 3 pts: agente com desempenho próximo ao agente de referência;
    - 4 pts: agente com desempenho superior ao agente de referência.
  - O objetivo dessa avaliação é premiar agentes que apresentem desempenho consistente ao longo de várias partidas.
  - Quantos aos recursos usados no torneio, o agente aleatório é exatamente o mesmo que vocês receberam. A coleção de músicas será diferente da que vocês receberam. Ela será, contudo, uma amostra da mesma coleção geral (de cerca de 146 mil letras de músicas brasileiras).
- Como sei se meu agente estratégico é realmente bom?
  - Compare a taxa de vitórias dele contra o agente aleatório.
  - Compare diferentes heurísticas umas com as outras.
  - Use os *logs* para entender o desempenho e ter ideias de como melhorar os **prompts**.
    - Além da taxa de vitórias, os logs são úteis para analisar qualidade das dicas, robustez do *parsing*, uso de *fallback* e consistência das decisões.
    - Em sistemas multiagente baseados em LLM local, a qualidade do agente depende não apenas do prompt, mas também de decisões de engenharia como *timeout*, *cache*, validação, *fallback* e formato das mensagens.

In [ ]:

## Observações Gerais

- A resposta deve ser entregue em trios.
- Plágio resultará em cancelamento da lista. Discutir soluções é permitido, mas a solução final deve ser própria.
- Data de entrega a ser definida no site da disciplina.